

Desafio de performance: Custo de ordenação

Rafael H. A. de Freitas¹

¹Faculdade da Saúde e Ecologia Humana (FASEH)
Rua São Paulo, 958, Parque Jardim Alterosa – CEP 33200-000 – Vespasiano – MG –
Brasil

{972320116}@ulife.com.br

Abstract. *This work presents a comparison between two algorithms aimed at solving the sorting problem. It explains the Bubble Sort and Heap Sort algorithms using text and pseudocode, and tests them to evaluate their performance with samples of different sizes and cases. The results demonstrate that Heap Sort, which is considered an efficient algorithm, achieved the best outcomes due to its complexity factor growing minimally, in contrast to Bubble Sort, which increases quadratically with the number of entries.*

Resumo. *Este trabalho apresenta uma comparação entre dois algoritmos com a finalidade de resolver o problema de ordenação. Nele, os algoritmos Bubble Sort e Heap Sort são explicados, usando texto e pseudocódigo, e colocados em teste, para averiguar seu desempenho com amostras de diferentes tamanhos e casos. O resultado demonstra que o Heap Sort, que um considerado um algoritmo eficiente, obteve os melhores resultados, pelo seu fator de complexidade crescer pouco, em relação ao Bubble Sort, que aumenta elevado ao quadrado do número de entradas.*

1. Introdução

Este trabalho tem como objetivo apresentar dois algoritmos de ordenação, um simples, sendo o *Bubble Sort*, e um eficiente, o *Heap Sort*. Algoritmos de ordenação cumprem a função de ordenar dados seguindo uma regra, seja ordem alfabética, ou em ordem crescente ao trabalhar com números, o qual foi a forma com que trabalhou-se nesse estudo.

O problema da ordenação consiste em ordenar dados e a forma com que isso é feito, porém, nesse trabalho expandimos o objetivo, também adicionando uma comparação entre os dois algoritmos, com testes para medir sua performance em seis casos diferentes. Trabalhando com um extenso array de números, foi utilizado marcadores de tempo no código, para verificar o tempo de execução de cada algoritmo em diferentes cenários, sendo esses: dados com o melhor caso, onde eles estão totalmente organizados; o caso médio, onde os dados estão aleatoriamente bagunçados; e o pior caso, onde os dados estão ordenados de forma decrescente. Nos três tipos de caso, houve duas variações na amostra, uma com mil números (1k), outra com um milhão de números (1M).

2. Funcionamento

2.1. Bubble Sort

Primeiro, vamos entender o funcionamento do Bubble Sort. Ele funciona baseado em repetições e trocas entre números vizinhos. Por exemplo: se o número na posição 1 for maior que o número da posição 2, a troca ocorre, e agora o código irá comparar o número da posição 2 com a posição 3. Ele percorre um array quantas vezes for necessário, toda vez que há uma troca, ele percorre mais uma vez; sendo simples, mas

muitas das vezes, ineficiente.

Um pseudocódigo para auxiliar a compreensão:

```
1 Enquanto (haver_troca = verdadeiro) faça:
2     houve_troca ← falso
3     Para (inteiro i = 0; i < largura do array; i++) faça:
4         Se (array[i] > array[i + 1]) faça:
5             variavel_armazenadora_temporaria ← array[i]
6             array[i] ← array[i + 1]
7             array[i + 1] ← variavel_armazenadora_temporaria
8             houve_troca ← verdadeiro
```

No pseudocódigo em questão, enquanto haver uma troca, o código ficará em um loop, percorrendo o array, e caso detecte a condição necessária para uma troca, ele a efetuará.

2.2. Heap Sort

O Heap Sort é baseado no funcionamento de árvores binárias, mas não igual, e usando arrays, gerando uma certa camada de abstração para se trabalhar com os dados. Ele funciona da seguinte maneira: primeiro pegasse o tamanho de itens no array, suponha que seja 10, o código irá pegar o tamanho e dividir por dois, e usar esse número resultante como índice do nó pai. Logo em seguida, ele obtém os filhos, da esquerda e direita, e compara os valores dos três elementos, caso um dos filhos seja menor que o pai, ele tem seu valor trocado com o pai. Agora, o filho que foi trocado se torna o pai, e a comparação continua. Caso não haja uma troca, se pega o valor do índice atual e se subtrai por 1. O código é executado até que o índice 0 seja subtraído por 1, por não haver troca.

Pseudocódigo [Guru99 2024]:

```
1  ALGORITMO HeapSort(array)
2      n ← comprimento(array)
3
4      // Fase 1: Construir o heap máximo
5      PARA i ← n/2 - 1 ATÉ 0 (decrescente)
6          AjustarHeap(array, n, i)
7      FIM PARA
8
9      // Fase 2: Extrair elementos do heap
10     PARA i ← n - 1 ATÉ 1 (decrescente)
11         // Trocar raiz (maior elemento) com último elemento
12         Trocar(array[0], array[i])
13
14         // Ajustar heap reduzido
15         AjustarHeap(array, i, 0)
16     FIM PARA
17
18     RETORNAR array
```

```

19 FIM ALGORITMO
20
21
22 ALGORITMO AjustarHeap(array, tamanho, indice)
23     maior ← indice
24     esquerda ← 2 * indice + 1
25     direita ← 2 * indice + 2
26
27     // Verificar se filho esquerdo é maior que a raiz
28     SE esquerda < tamanho E array[esquerda] > array[maior] ENTÃO
29         maior ← esquerda
30     FIM SE
31
32     // Verificar se filho direito é maior que o atual maior
33     SE direita < tamanho E array[direita] > array[maior] ENTÃO
34         maior ← direita
35     FIM SE
36
37     // Se o maior não é a raiz, trocar e recursivamente ajustar
38     SE maior ≠ indice ENTÃO
39         Trocar(array[indice], array[maior])
40         AjustarHeap(array, tamanho, maior)
41     FIM SE
42 FIM ALGORITMO
43
44
45 ALGORITMO Trocar(elemento1, elemento2)
46     temporario ← elemento1
47     elemento1 ← elemento2
48     elemento2 ← temporario
49 FIM ALGORITMO

```

3. Resultados

Resultados das diferentes execuções dos algoritmos. A primeira tabela se refere ao Bubble Sort, a segunda ao Heap Sort.

Tabela 1. Resultados do Bubble Sort

<i>Tipo de caso</i>	<i>Tamanho da amostra</i>	<i>Tempo de execução em milsegundos</i>
Melhor	Mil itens	1
Melhor	Um milhão de itens	6
Médio	Mil itens	7
Médio	Um milhão de itens	1475364 (cerca de 24 min)
Pior	Mil itens	7
Pior	Um milhão de itens	654990 (cerca de 10 min)

Tabela 2. Resultados do Heap Sort

<i>Tipo de caso</i>	<i>Tamanho da amostra</i>	<i>Tempo de execução em milsegundos</i>
Melhor	Mil itens	1
Melhor	Um milhão de itens	76
Médio	Mil itens	1
Médio	Um milhão de itens	79
Pior	Mil itens	2
Pior	Um milhão de itens	74

4. Análise crítica de complexidade

O Bubble Sort é um algoritmo com complexidade $O(n^2)$, o que significa que o tempo de execução cresce quadraticamente com o tamanho da entrada. Por exemplo, enquanto ao usar a amostra com 1 milhão de itens, será necessário cerca de 1 trilhão de operações até o fim da ordenação, comparado ao Heap Sort, com complexidade $O(n \log n)$, com cerca de 20 milhões de operações – 50 mil vezes menor que o Bubble. Sendo assim, o Bubble Sort demorou muito mais no arquivo grande por ser um número gigante de entradas elevado ao quadrado.

O crescimento do tempo de execução do Heap Sort é mais controlado, quando você aumenta o tamanho da entrada, o tempo de execução cresce de forma logarítmica em relação ao tamanho. O fator $\log n$ cresce muito lentamente. Quando você vai de 1 milhão para 10 milhões de elementos:

- n aumenta 10 vezes
- $\log n$ aumenta apenas de ~20 para ~23 (1,15 vezes maior)

5. Conclusão

Este estudo comparou dois algoritmos de ordenação, o Bubble Sort e o Heap Sort, demonstrando claramente as diferenças em eficiência entre eles. Ao longo da análise, ficou evidente que, embora o Bubble Sort tenha uma implementação simples e intuitiva, sua complexidade de tempo $O(n^2)$ torna-o inadequado para grandes volumes de dados. Em contrapartida, o Heap Sort, com uma complexidade de tempo $O(n \log n)$, se mostrou significativamente mais eficiente em todos os casos testados, especialmente em amostras de grande dimensão.

Os resultados obtidos foram consistentes com a expectativa teórica: o Heap Sort conseguiu concluir a ordenação em tempos drasticamente menores do que o Bubble Sort, mesmo quando operando com um milhão de entradas. Ao medir o tempo de execução em diferentes cenários, observaram-se aumentos controlados no desempenho do Heap Sort em comparação aos saltos exponenciais do Bubble Sort.

Esta comparação enfatiza a importância de escolher o algoritmo de ordenação adequado em função do tamanho e da natureza dos dados a serem processados. A pesquisa não apenas fornece uma compreensão clara do funcionamento interno de ambos os algoritmos, mas também serve como um alerta para a escolha de métodos eficientes em aplicações que requerem processamento de dados em larga escala. Em futuras investigações, seria interessante explorar outros algoritmos de ordenação e suas performances em diferentes contextos, além de considerar aspectos como a estabilidade dos algoritmos e a sua aplicabilidade em dados realísticos.

Referencias

GURU99. *Heap Sort em Java: Algoritmo e Exemplo*. Disponível em: <https://www.guru99.com/pt/heap-sort.html>. Acesso em: 29 maio 2026.